

ENTWURF: PARALLELER IMAGE CRAWLER

Ziel des Programms

Das Programm implementiert einen einfachen Image Crawler. Er lädt eine übergebene Webseite, durchsucht deren HTML-Quelltext nach Bildern und speichert die gefundenen Bilddateien lokal ab. Der Schwerpunkt liegt dabei nicht auf einem möglichst vollständigen Webbrowser, sondern auf einer klar abgegrenzten, parallelisierten Verarbeitung: HTML laden, Bildadressen erkennen und Bilder herunterladen.

Der Crawler beschränkt sich bewusst auf normale HTML-Seiten mit `<img>`-Tags. Dynamisch per JavaScript nachgeladene Inhalte werden nicht berücksichtigt. Ebenso folgt der Crawler keinen Links auf weitere HTML-Seiten. Dadurch bleibt der fachliche Umfang überschaubar und der Fokus liegt auf der geforderten Parallelisierung.

Für jeden Aufruf von `crawl(uri)` wird unter dem konfigurierten Download-Pfad ein eigener Unterordner angelegt. Diese Ordner werden fortlaufend nummeriert, beginnend bei `1`. So lässt sich später eindeutig erkennen, welche Dateien zu welchem Crawl-Aufruf gehören. Die Nummerierung ist auch dann korrekt, wenn mehrere Threads gleichzeitig neue Webseiten an den Crawler übergeben.

Aufbau und Verantwortlichkeiten

Die zentrale Klasse des Programms ist `ImageCrawler`. Sie nimmt neue Crawl-Aufträge entgegen, vergibt die Zielordner und koordiniert die Arbeit zwischen Webseitenanalyse und Bilddownload. Die eigentlichen fachlichen Schritte sind bewusst in eigene Klassen ausgelagert.

`WebsiteAnalyzer` ist für die Analyse der Webseiten zuständig. Die Klasse lädt die HTML-Seite mit dem Java-`HttpClient`, parst den Inhalt mit Jsoup und extrahiert die Bildadressen aus den gefundenen ``-Elementen. Relative Bildpfade werden dabei gegen die ursprüngliche Webseitenadresse aufgelöst. Ungültige oder nicht unterstützte Adressen, zum Beispiel `data:`- oder `ftp:`-Links, werden übersprungen.

`ImageDownloader` übernimmt anschließend den Download der Bilddateien. Er lädt die Daten über HTTP oder HTTPS und speichert sie im passenden Zielordner. Die Dateinamen werden durch `FileNameResolver` vorbereitet. Diese Trennung hält die Download-Klasse übersichtlich und kapselt die Sonderlogik für doppelte Dateinamen an einer Stelle.

Wenn mehrere Bilder im selben Zielordner denselben Namen tragen, bleibt die erste Datei unverändert. Weitere Dateien erhalten einen Suffix vor der Dateiendung, zum Beispiel `image_2.jpg` oder `image_3.jpg`. Die Reservierung des Zielpfads passiert bereits vor dem Start der Download-Aufgabe. Dadurch hängt die Benennung nicht davon ab, welcher Bildserver zufällig schneller antwortet.

`ImageReference` dient als kleine Datenklasse für einzelne Download-Aufgaben. Sie enthält die Bild-URI, den Zielordner und bei Bedarf den bereits reservierten Zielpfad.

Datenfluss

Der Ablauf beginnt mit einem Aufruf von `crawl(uri)`. `ImageCrawler` bestimmt zuerst den nummerierten Zielordner und legt anschließend eine Aufgabe für die Webseitenanalyse an. Diese Aufgabe wird an den Website-Executor übergeben.

Sobald ein Website-Worker frei ist, ruft er `WebsiteAnalyzer.analyze(uri)` auf. Das Ergebnis ist eine Liste von Bild-URIs. Für jede gefundene Bildadresse reserviert der Crawler einen eindeutigen Zielpfad und erstellt eine `ImageReference`. Diese Referenzen werden danach als eigene Aufgaben an den Download-Executor weitergereicht.

Der Datenfluss lässt sich in fünf Schritte aufteilen:

1. Webseite entgegennehmen und Zielordner festlegen
2. HTML-Seite laden und `<img>`-Tags auswerten
3. Bildadressen in Download-Aufgaben umwandeln
4. eindeutige Zielpfade reservieren
5. Bilder parallel herunterladen und speichern

Vereinfacht entsteht damit folgende Pipeline:

```
```text
crawl(uri)
 -> ImageCrawler vergibt Zielordner
 -> WebsiteAnalyzer lädt und analysiert HTML
 -> ImageCrawler erzeugt ImageReference-Aufgaben
 -> ImageDownloader lädt und speichert Bilder
```
```

Parallelisierung

Die Webseitenanalyse und die Bilddownloads werden unabhängig voneinander parallelisiert. Dafür besitzt `ImageCrawler` zwei feste Threadpools. Der erste Threadpool verarbeitet nur Webseiten-Scans. Seine Größe wird über `getNumberOfAllowedParallelWebsiteScans()` konfiguriert. Der zweite Threadpool verarbeitet nur Bilddownloads. Seine Größe wird über `getNumberOfAllowedParallelImageDownloads()` festgelegt.

Diese Trennung ist wichtig, weil beide Arbeitsschritte unterschiedliche Wartezeiten haben können. Ein langsamer Bilddownload soll nicht verhindern, dass weitere Webseiten analysiert werden. Gleichzeitig sollen viele gefundene Bilder nicht dazu führen, dass unbegrenzt viele Downloads parallel starten. Die internen Warteschlangen der Executor-Services übernehmen das geordnete Zurückstellen von Aufgaben, wenn alle erlaubten Worker gerade beschäftigt sind.

Als algorithmisches Muster wird Task Parallelism verwendet: Jede Webseitenanalyse und jeder Bilddownload ist eine eigene Aufgabe. Zusätzlich entsteht eine einfache Pipeline, weil die Webseitenanalyse Bildadressen produziert, die anschließend von der Download-Stufe verarbeitet werden. Aus Sicht der Implementierung entspricht die Struktur einem Master/Worker-Ansatz: `ImageCrawler` nimmt Arbeit an, hält den Zustand nach und verteilt Aufgaben an Worker-Threads.

Thread-Safety

Mehrere Teile des Programms werden von verschiedenen Threads verwendet und müssen deshalb geschützt werden. Die Nummerierung der Zielordner erfolgt über einen `AtomicInteger`. Dadurch kann auch bei gleichzeitigen `crawl`-Aufrufen keine Nummer doppelt vergeben werden.

Auch der Arbeitszustand des Crawlers wird über atomare Zähler verfolgt. Es gibt jeweils Zähler für wartende und laufende Webseiten-Aufgaben sowie für wartende und laufende Bild-Aufgaben. `isIdle()` liest diese Werte aus und gibt nur dann `true` zurück, wenn in beiden Stufen keine Arbeit mehr offen ist.

Bei der Dateinamenvergabe wird ein kleiner synchronisierter Bereich verwendet. `FileNameResolver.resolveTargetPath(...)` muss pro Zielordner wissen, welche Namen bereits reserviert wurden. Nur diese Reservierung ist synchronisiert. Der eigentliche HTTP-Download und das Schreiben der Datei liegen außerhalb des Locks. Damit bleibt der Schutz auf den wirklich gemeinsamen Zustand beschränkt.

Fehlerbehandlung

Fehler einzelner Webseiten oder Bilder beenden den Crawler nicht. Wenn eine Webseite einen HTTP-Fehlerstatus liefert, entsteht einfach keine Bildliste. Schlägt ein Webseiten-Request fehl, wird nur diese eine Webseiten-Aufgabe beendet. Wenn ein Bild nicht geladen werden kann, wird nur dieser einzelne Download ignoriert.

Diese Fehlerbehandlung passt zum Ziel der Aufgabe. Der Schwerpunkt liegt auf der Parallelisierung und nicht auf einer vollständigen Behandlung aller möglichen Besonderheiten im Web. Ungültige Bildlinks, fehlende Dateiendungen oder ungewohnte Dateinamen werden daher einfach und robust behandelt, ohne die übrige Verarbeitung zu blockieren.